

Vantage — Metrics Reference

Purpose: Explains the nine performance metrics, the peer average, the vendor ranking, and the safeguards that keep AI-generated narrative consistent with them. Intended for technical review, and as the single source of truth that the Test Plan Document (TPD) and User Acceptance Test (UAT) document refer to.

Source of truth: the code in `lib/domain/reports/`. Every formula below was read from that code, and the worked example in section 7 was run against it.

Related documents: URD (FR-M1-004 to FR-M1-008, NFR-010), SDD (Metric Engine, P3 to P5).

1. Summary

Principle	Meaning
Deterministic first	Application code computes every metric and every ranking. The AI model never calculates anything.
AI explains only	Gemini receives the computed values and writes narrative around them.
Validated output	Narrative containing a number that is not in the supplied data is rejected and regenerated (up to 3 attempts).
No data is not zero	A metric with no eligible transactions is left undefined (NULL), never shown as 0.

In plain terms: the software does the maths, the AI only writes the commentary, and the software checks the commentary against its own figures before showing it.

2. Reporting periods

- **Current period:** a quarter or a custom date range chosen by the user.
 - **Prior period:** the period of equal length that ends the day before the current period starts (`getPriorPeriod.ts`).
 - Example: current 2026-01-01 to 2026-03-31 (90 days) gives prior 2025-10-03 to 2025-12-31 (90 days).
 - Each metric is computed for the **current period**, the **prior period**, and as a **peer average** (section 4).
-

3. The nine metrics

Metrics are grouped into three benchmarks. Each metric only uses transactions that have the data it needs (its **eligible** transactions). A transaction missing the relevant field is excluded from that metric. It is not treated as a failure or as zero. Results are rounded to 2 decimal places.

3.1 Delivery timing

Eligible: transactions with an actual delivery date. A transaction not yet delivered is excluded.

#	Metric	Formula	Unit	Better
1	On-time delivery rate	$(\text{deliveries where actual date is on or before agreed date}) / (\text{eligible deliveries}) \times 100$	%	Higher
2	Average delay	Mean of (actual date minus agreed date, in days) over late deliveries only (actual date after agreed date)	days	Lower

Notes: - Delivering on the agreed date counts as on time. - Early deliveries are not counted as negative delay. They are left out of the average delay. - If there are eligible deliveries but none is late, average delay is **0** (a measured zero). If there are no eligible deliveries, it is **undefined**.

3.2 Pricing consistency

Eligible: transactions with an actual price.

#	Metric	Formula	Unit	Better
3	Overcharge rate	$(\text{transactions where actual price is above agreed price}) / (\text{eligible}) \times 100$	%	Lower
4	Average overcharge	Mean of $((\text{actual minus agreed}) / \text{agreed} \times 100)$ over overcharged transactions only	%	Lower
5	Undercharge rate	$(\text{transactions where actual price is below agreed price}) / (\text{eligible}) \times 100$	%	No direction (see below)

Notes: - Average overcharge is 0 when there are priced transactions but none overcharged; undefined when there are no priced transactions. - **Undercharge rate is informational**. A vendor charging less than agreed benefits the buyer, so it carries no good or bad judgment. It is shown in the metrics table but excluded from colour coding and from ranking.

3.3 Order accuracy

Eligible: transactions with a recorded quantity received.

#	Metric	Formula	Unit	Better
6	Shortfall rate	$(\text{received below ordered}) / (\text{eligible}) \times 100$	%	Lower
7	Average shortfall	Mean of (ordered minus received) over short transactions only	units	Lower
8	Over-delivery rate	$(\text{received above ordered}) / (\text{eligible}) \times 100$	%	Lower
9	Average over-delivery	Mean of (received minus ordered) over over-delivered transactions only	units	Lower

Notes: - Over-delivery is treated as a deviation from the order (lower is better), not as a bonus. - Averages are 0 when eligible transactions exist but none deviates; undefined when none are eligible.

3.4 Why "rate" and "average" behave differently

A **rate** divides by all eligible transactions (how often does it happen?). An **average** divides only by the transactions where it happened (how bad is it when it happens?). So a vendor can have a low rate but a high average, which is why both are reported.

In plain terms: the rate says how often a vendor slips up; the average says how big the slip is each time.

4. Peer average

For each metric and each vendor, the **peer average** is the simple mean of that metric's current-period value across **all other vendors in the same report** (`calculatePeerAverage.ts`).

- The vendor itself is excluded.
 - Vendors whose value is undefined are skipped, so they do not drag the average towards zero.
 - If no peer has a value, the peer average is undefined.
 - It is a mean of vendor-level values, not a pool of all peers' transactions. A vendor with 5 transactions counts the same as one with 500.
 - A report with one vendor has no peers, so every peer average is undefined.
-

5. Vendor ranking (reports with 2 or more vendors)

Ranking is computed by code (`rankVendorComparison.ts`) on **current-period** metrics only. The AI never ranks.

1. **Per metric:** vendors are ranked independently on each of the eight directional metrics, using the "Better" direction in section 3. Vendors with an undefined value are left out of that metric's ranking.
2. **Per category:** a vendor's category score is the average of its own metric ranks. - Delivery: on-time delivery rate, average delay - Pricing: overcharge rate, average overcharge - Order accuracy: shortfall rate, average shortfall, over-delivery rate, average over-delivery - Vendors are then ranked by category score.
3. **Overall:** the average of the vendor's three category ranks, with equal weight. A vendor must be ranked in all three categories to get an overall rank; otherwise its overall result is "not enough data".
4. **Ties:** equal scores share a rank and the next rank is skipped ("1, 1, 3"). Tied results are flagged. Scores are rounded to 4 decimals before comparison so floating-point noise cannot break a true tie.

Two things a reviewer should know: - Ranking uses **ranks**, not the size of differences. A vendor 0.1 points behind and one 30 points behind both simply rank lower. - The good/warning/danger colour bands (e.g. on-time at least 90% is good, below 75% is danger) are **proposed defaults**, not requirements. They are in `lib/ui/reports/getMetricSeverity.ts` and affect display only. The ranking reads only the direction from that file, not the thresholds.

6. How the AI is kept consistent with the metrics

Step	What happens	Where
1	Computed values (current, prior, peer average per metric, plus vendor names) are sent to Gemini as structured data.	<code>buildMetricPromptData.ts</code> , <code>runReportGeneration.ts</code>
2	The prompt instructs the model to use only supplied numbers, to quote them exactly or round to a whole number, and not to state a number for a null field.	<code>generateReportNarrative.ts</code>
3	Number check: every number in the narrative must equal a supplied value (to 2 decimals) or its nearest whole number. Dates and "Vendor N" references are ignored.	<code>validateNarrativeNumbers.ts</code>
4	Comparison check: wording such as "improving" or "worse" must be accompanied by a reference to both the prior period and the peer average.	<code>validateNarrativeComparisons.ts</code>
5	Leader check: AI comparison text may not name a vendor as leading or winning unless the computed ranking says so.	<code>validateVendorComparisonNarrative.ts</code>
6	If any check fails, generation retries, up to 3 attempts. If all fail, generation returns an error and nothing is saved.	<code>runReportGeneration.ts</code>
7	The ranking statement itself (who leads) is written by code, shown before the AI text.	<code>formatVendorComparisonSummary.ts</code>

Known limitations

These are honest boundaries of the safeguards, worth stating to a reviewer:

1. **The number check is value-based, not field-based.** It confirms a number exists somewhere in the supplied data, not that it belongs to the metric or vendor being described. A narrative that attaches the right number to the wrong vendor would pass this check.
2. **The comparison and leader checks are keyword heuristics.** They match listed words and phrases, and cannot understand negation ("does not lead") or unusual phrasing.
3. **Rounding to whole numbers is accepted by design**, since natural text rounds. A narrative may say "86%" for 85.71%.
4. **Colour bands are proposed defaults**, not signed-off business thresholds.

7. Worked example

One vendor, four transactions in January 2026 (the last is not yet delivered or priced).

Txn	Agreed delivery	Actual delivery	Agreed price	Actual price	Ordered	Received
T1	10 Jan	8 Jan	100	110	100	90
T2	15 Jan	15 Jan	200	200	50	50
T3	20 Jan	23 Jan	50	45	20	25
T4	25 Jan	(none)	80	(none)	40	(none)

Metric	Working	Result
On-time delivery rate	Eligible T1-T3 (T4 excluded). On time: T1, T2. 2 of 3	66.67%
Average delay	Late deliveries: T3 only (3 days)	3.00 days
Overcharge rate	Priced T1-T3. Overcharged: T1. 1 of 3	33.33%
Average overcharge	T1: (110 - 100) / 100	10.00%
Undercharge rate	Undercharged: T3. 1 of 3	33.33%
Shortfall rate	Recorded T1-T3. Short: T1. 1 of 3	33.33%
Average shortfall	T1: 100 - 90	10.00 units
Over-delivery rate	Over: T3. 1 of 3	33.33%
Average over-delivery	T3: 25 - 20	5.00 units

(Computed by the application code: all nine values match.)

Ranking example. Add a second vendor with on-time 80%, average delay 1 day, overcharge rate 50%, average overcharge 12%, and zero shortfall and over-delivery.

	Delivery	Pricing	Order accuracy	Overall
Vendor 1 (above)	2nd	1st	2nd	2nd
Vendor 2	1st	2nd	1st	1st

Vendor 2 leads overall because it ranks first in two of three categories, while Vendor 1 leads on pricing.

8. Traceability for the TPD and UAT

Use this table to write test cases. "Unit test" files already exist next to each source file.

Topic	Source file	Existing unit test	Suggested UAT check
On-time delivery rate	<code>calculateOnTimeDeliveryRate.ts</code>	<code>.test.ts</code> alongside	Upload transactions with early, on-time, late and undelivered rows; confirm the rate excludes undelivered
Average delay	<code>calculateAverageDelayDays.ts</code>	<code>.test.ts</code> alongside	Confirm early deliveries do not reduce the delay; all on-time shows 0
Overcharge, average overcharge, undercharge	<code>calculateOverchargeRate.ts</code> , <code>calculateAverageOverchargePct.ts</code> , <code>calculateUnderChargeRate.ts</code>	<code>.test.ts</code> alongside	Mix of over, equal, under and missing price rows
Shortfall and over-delivery (4 metrics)	<code>calculateShortfallRate.ts</code> , <code>calculateAverageShortfallUnits.ts</code> , <code>calculateOverdeliveryRate.ts</code> , <code>calculateAverageOverdeliveryUnits.ts</code>	<code>.test.ts</code> alongside	Mix of short, exact, over and missing quantity rows

Topic	Source file	Existing unit test	Suggested UAT check
No data is undefined, not zero	all calculators	<code>.test.ts</code> alongside	A vendor with no transactions in the period shows a dash or "no data", not 0
Prior period	<code>getPriorPeriod.ts</code>	<code>getPriorPeriod.test.ts</code>	Quarter and custom range; confirm equal length and adjacent dates
Peer average	<code>calculatePeerAverage.ts</code>	<code>calculatePeerAverage.test.ts</code>	Two, three and one-vendor reports; one peer with no data
Ranking and ties	<code>rankVendorComparison.ts</code>	<code>rankVendorComparison.test.ts</code>	Two vendors with a deliberate tie; vendor with missing category
Narrative number check	<code>validateNarrativeNumbers.ts</code>	<code>validateNarrativeNumbers.test.ts</code>	Compare every figure in a generated narrative to the metrics table
Narrative comparison and leader checks	<code>validateNarrativeComparisons.ts</code> , <code>validateVendorComparisonNarrative.ts</code>	<code>matching.test.ts</code>	Read the narrative: any "improving" or "leads" claim is backed by the figures
Retry and failure	<code>runReportGeneration.ts</code>	<code>runReportGeneration.test.ts</code>	Not repeatable by hand in UAT; cover in the TPD with a mocked model

Requirement mapping: FR-M1-004 (metrics and NULL handling), FR-M1-005 (peer average), FR-M1-006 (structured input, no invented numbers), FR-M1-008 (comparison before characterization), NFR-010 (deterministic computation).